

SEMINARARBEIT

Rahmenthema des Wissenschaftspropädeutischen Seminars:

Klima

Leitfach: Physik

Thema der Arbeit:

Bearbeitung der 2. Runde des 43.
Bundeswettbewerb-Informatik

Verfasser/in: Quirin Klemm

Kursleiter/in: Ulrich Steiner

Abgabetermin:

11. November 2025

Bewertung	Note	Notenstufe in Worten	Punkte		Punkte
Schriftliche Arbeit				x 3	
Abschlusspräsentation				x 1	
Summe:					
Gesamtleistung nach § 29 (6) GSO = Summe : 2 (gerundet)					

Datum und Unterschrift der Kursleiterin oder des Kursleiters

Erklärung:

Ich versichere, dass ich die vorgelegte Seminararbeit persönlich und unverfälscht verfasst, sämtliche hierfür zu Hilfe genommene gedruckte sowie digitale Quellen im Literaturverzeichnis angegeben und die aus diesen Quellen stammenden Zitate oder Belegstellen für sinngemäß wiedergegebene Inhalte in meiner Seminararbeit als solche kenntlich gemacht habe.

Die Seminararbeit ist in dieser oder einer ähnlichen Form in keinem anderen Kurs des diesjährigen oder eines vorhergehenden Abiturjahrgangs vorgelegt worden.

.....
Ort, Datum

.....
Unterschrift des/der Oberstufenschülers/in

S E M I N A R A R B E I T

Rahmenthema des Wissenschaftspropädeutischen Seminars:

Klima

Leitfach: Physik

Thema der Arbeit:

Bearbeitung der 2. Runde des 43.
Bundeswettbewerb-Informatik

Verfasser/in: Quirin Klemm

Kursleiter/in: Ulrich Steiner

Inhaltsverzeichnis

1	Einleitung	3
2	Aufgabe 1: Schmucknachrichten	3
2.1	Problemstellung	3
2.2	Theoretische Erklärung des Algorithmus	4
2.2.1	Betrachtung von Symbolen mit gleichen Kosten	4
2.2.2	Erweiterung auf Symbole mit unterschiedlichen Kosten	6
2.3	Implementierung des Algorithmus	7
2.3.1	Erstellung des Baums	8
2.3.2	Permutation des Baums	9
2.4	Laufzeitanalyse und Einordnung des Algorithmus	10
2.4.1	Analyse des Algorithmus	10
2.4.2	Stand der Forschung	11
2.4.3	Fazit	12
2.5	Vergleich mit der Musterlösung	12
3	Aufgabe 2: Simultane Labyrinth	13
3.1	Problemstellung	13
3.2	Theoretische Erklärung des Algorithmus	13
3.2.1	Erstellung der Matrix	13
3.2.2	Lösen der Labyrinth	14
3.3	Implementierung des Algorithmus	15
3.3.1	Erstellung der Matrix	15
3.3.2	Lösen der Labyrinth	16
3.4	Laufzeitanalyse	17
3.5	Vergleich mit der Musterlösung	20
4	Fazit	20
5	Anhang	21
5.1	Lösungen der Testfälle	21
5.1.1	Aufgabe 1: Schmucknachrichten	21
5.1.2	Aufgabe 2: Simultane Labyrinth	22
5.2	Quellcode	22
5.2.1	Aufgabe 1: Schmucknachrichten (gekürzt)	22
5.2.2	Aufgabe 2: Simultane Labyrinth (gekürzt)	25
5.3	Quellen	30

1 Einleitung

Die vorliegende Arbeit ist meine Einreichung für die zweite Runde des 43. Bundeswettbewerb Informatik. Es sind zwei Aufgaben zu bearbeiten, die jeweils ein Problem beschreiben. Für jedes Problem soll ein Algorithmus entwickelt werden, mit dem sich dieses lösen lässt. Anschließend muss der Algorithmus auf verschiedene Testfälle angewandt werden, damit die Korrektoren die Richtigkeit und die Genauigkeit beurteilen können. Die Teilnehmenden kennen dabei die erwarteten Ergebnisse der Testfälle nicht. Die ausgearbeiteten Lösungen werden anschließend nach einem festgelegten Schema korrigiert und benotet. Pro Aufgabe können bis zu 20 Punkte erreicht werden, wobei gute Ausarbeitungen Zusatzpunkte erzielen und unvollständige Bearbeitungen Punktabzüge nach sich ziehen.[1] Die Seminararbeit ist in zwei Teile gegliedert, beginnend mit Aufgabe 1, Schmucknachrichten, und daraufhin Aufgabe 2, simultane Labyrinth.

Zuerst erkläre ich jeweils kurz die Aufgabenstellung und beschreibe meine theoretischen Überlegungen. Im Anschluss zeige ich, wie sie praktisch in Python3 umgesetzt werden. Zuletzt diskutiere ich meine Ergebnisse und vergleiche meine Lösung mit der offiziellen Musterlösung.

2 Aufgabe 1: Schmucknachrichten

2.1 Problemstellung

In dieser Aufgabe möchten Sybille und Pedram kurze Nachrichten, die aus verschiedenen Buchstaben bestehen, in Perlen-Codes für Ketten umwandeln. Für jeden Buchstaben ist ein Codewort aus Perlen zu finden, wobei insgesamt ein präfixfreier Code entstehen muss. Dabei soll die Gesamtlänge der codierten Nachricht möglichst gering sein.[1] Daher wird ein präfixfreier Code gesucht, bei dem häufig vorkommende Buchstaben kurze Codewörter erhalten, während seltenere Buchstaben längere Codewörter zugewiesen bekommen. „Ein Code C heißt Präfix-Code (irreduzibler Code), wenn kein Codewort aus C Präfix eines anderen Codeworts von C ist.“[8, S. 37] Dadurch lässt sich jede Nachricht eindeutig codieren und wieder decodieren.[8, S. 37] Ein Code mit den Codewörtern $\{0, 10, 11\}$ ist präfixfrei, da kein Codewort der Anfang eines anderen ist. Im Gegensatz dazu ist der Code $\{0, 01, 11\}$ nicht präfixfrei, weil „0“ das Präfix von „01“ ist. Zusätzlich wird in dieser Aufgabe zwischen zwei Varianten unterschieden. In der ersten Teilaufgabe haben alle Perlen denselben Durchmesser, während in der zweiten Teilaufgabe die Perlen unterschiedliche Durchmesser besitzen.[1]

Im Folgenden wird der Begriff Perlen durch den allgemeineren Begriff Symbole ersetzt.

2.2 Theoretische Erklärung des Algorithmus

2.2.1 Betrachtung von Symbolen mit gleichen Kosten

In meiner Arbeit gilt für die Kosten der codierten Nachricht, solange alle Symbole die gleichen Kosten haben:

$$c_i = p_i \cdot n_i \quad (1)$$

$$l = \sum_i c_i \quad (2)$$

wobei gilt:

- c_i die Kosten des Buchstabens i
- p_i die Häufigkeit des Buchstabens i in der Nachricht
- n_i die Länge des Codewortes für den Buchstaben i (Anzahl der Symbole)
- l die Länge der codierten Nachricht

Um einen präfixfreien Code zu erzeugen, wird ein Baum verwendet. In diesem entspricht jedes Blatt einem Codewort. Die Kanten auf dem Weg von der Wurzel zu einem Blatt repräsentieren dabei die Symbole des jeweiligen Codewortes. Diese präfixfreie Eigenschaft ergibt sich automatisch aus der Baumstruktur. Das heißt, sobald ein Blatt erreicht wird, endet das Codewort, und keine weiteren Kanten bzw. Symbole können daran angeschlossen werden. Der Baum hat mindestens so viele Blätter, wie es verschiedene Buchstaben gibt, da jeder Buchstabe sein eigenes Codewort braucht. Jeder innere Knoten kann dabei maximal so viele Kinder haben, wie es verschiedene Symbole gibt. Die Ebene des Blatts entspricht der Länge des Codeworts.

Jeder präfixfreie Code kann in einem Baum dargestellt werden und jeder Baum kann einen präfixfreien Code darstellen.[8, S. 37] Daher kann das Problem auf das Finden eines optimalen Baumes reduziert werden.

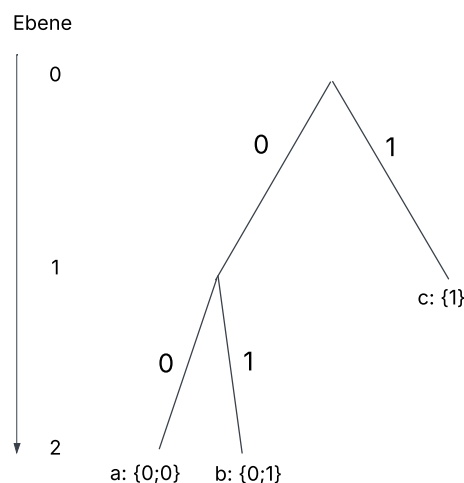


Abbildung 1: einfacher Baum

Um einen möglichst kleinen präfixfreien Code zu erstellen, muss der Buchstabe, der am häufigsten vorkommt, ganz oben eingeordnet werden und die nächsten Buchstaben dann in absteigender Reihenfolge. In dem in Abbildung 1 dargestellten Baum hat c den kürzesten Code und sollte damit optimalerweise der häufigste Buchstabe sein.

Daher ergibt sich $p_1 \geq p_2 \geq p_3 \dots \geq p_i$ und $n_1 \leq n_2 \leq n_3 \dots \leq n_i$

Zu finden ist also ein Baum, der basierend auf den Häufigkeiten der Buchstaben seine Blätter so auf seine Ebenen verteilt, dass die Länge der codierten Nachricht nach Gleichung (2) möglichst gering ist. Dabei muss jeder innere Knoten mindestens zwei Kinder haben, da ein innerer Knoten mit nur einem Kind die Zweige des Baums verlängert, ohne einen Vorteil zu bringen.

Zu Beginn des Algorithmus wird ein balancierter Baum erstellt, der so viele Blätter wie Buchstaben hat. (vgl. Abbildung 2) Jedes Blatt wird von links nach rechts in aufsteigender Reihenfolge der Häufigkeit mit einem Buchstaben belegt, und die Gesamtkosten des Baums werden nach Formel (2) berechnet. Dabei ergeben sich deutlich höhere Kosten für die rechten Buchstaben im Baum im Vergleich zu den linken, da alle Blätter auf derselben Ebene sind, jedoch die Häufigkeit der Buchstaben auf der rechten Seite größer ist.

Um dies auszugleichen, wird der Buchstabe mit den höchsten Kosten eine Ebene nach oben verschoben, indem der Elternknoten des Blattes selber zum Blatt wird und alle Kinder verliert. Dadurch verliert der Baum aber zu viele Kinder, weshalb er erweitert werden muss. Für die Erweiterung untersucht der Algorithmus, an welche Knoten neue Blätter angehängt werden können. Er überprüft die Blätter nach ihrer Ebene, d.h. nach ihrem Wert für n , und fügt das mit dem kleinsten Wert hinzu. Das nach oben verschobene Blatt erhält den Zusatz *protected*. Das verhindert, dass es selbst weitere Kinder bekommen kann. Dadurch wird ein Zyklus vermieden, bei dem ein Blatt zunächst nach oben verschoben und anschließend wieder nach unten erweitert werden würde.

Nach jeder Iteration werden wie schon zu Beginn den Blättern Buchstaben aufsteigend zugewiesen. Durch die Neuzuweisung ist sichergestellt, dass immer die äußeren linken Blätter die Buchstaben mit den geringsten Häufigkeiten bekommen und die Blätter rechts die mit den höchsten.

Der Vorgang, in dem ein Blatt nach oben geschoben wird und unten neue Blätter hinzugefügt werden, wird so lange fortgesetzt, bis keine neuen Blätter mehr eingefügt werden können, da alle Blätter als *protected* gelten. Das ist der Fall kurz vor einem maximal ausgearteten Baum (Abbildung 5). Der Algorithmus geht also von einem Maximum (maximal ausbalanciert) zum anderen Maximum (maximal ausgeartet). Am Ende wird der Baum mit den geringsten Gesamtkosten ausgegeben.

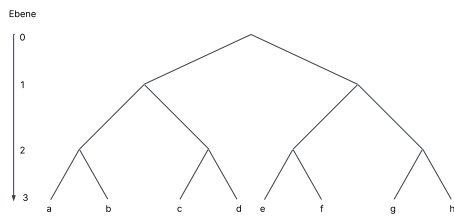


Abbildung 2: Balancierter Baum

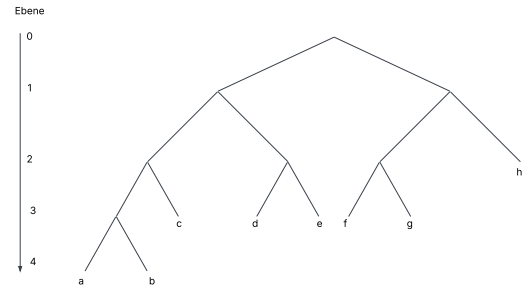


Abbildung 3: Transformation 1

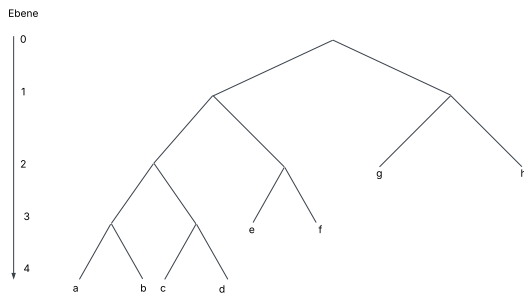


Abbildung 4: Transformation 2

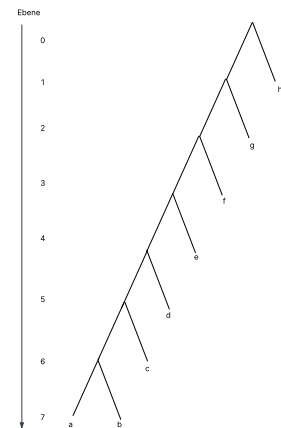


Abbildung 5: Ausgearteter Baum

2.2.2 Erweiterung auf Symbole mit unterschiedlichen Kosten

Das Problem bei Symbolen mit unterschiedlichen Kosten besteht darin, dass Blätter auf derselben Ebene unterschiedliche Kosten für die Gesamtlänge der Nachricht haben können. Daher entsprechen die Kosten nicht mehr dem Wert der Ebene. Das hat zur Folge, dass die Gleichung (1) nicht mehr gilt, stattdessen müssen die Kosten jedes Symbols im Codewort einzeln betrachtet werden. Daher ergibt sich

$$c_i = p_i \cdot \sum_j s_j \quad (3)$$

wobei gilt:

- s_j die Kosten des Symbols j
- j iteriert dabei durch die Liste der Symbole des Codewortes i

Aus diesem Grund stellt man nun Bäume nicht nach ihrer Ebene, sondern nach ihren Kosten dar. Diese werden auch in der Literatur als *lopsided tree*[5, S. 4] bezeichnet (Abbildung 6).

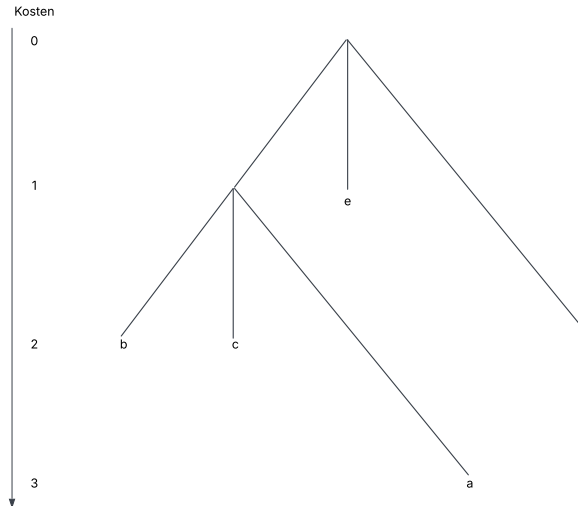


Abbildung 6: lopsided tree: Perlendurchmesser: (1; 1; 2)

In diesem Fall gilt weiterhin, jeder innere Knoten muss mindestens zwei Kinder haben. Bei dieser Art von Bäumen hat jedoch nicht der äußerste rechte Knoten die geringsten Kosten, sondern der oberste. Der Algorithmus bleibt unverändert, d.h. zunächst wird ein Baum erstellt, in dem alle Blätter ungefähr die gleichen Kosten haben. Anschließend wird der Buchstabe mit den höchsten Kosten nach oben verschoben, während der Baum unten erweitert wird. Dies geschieht solange, bis kein Blatt mehr eingefügt werden kann und der beste Baum ausgegeben wird.

Wenn die Kosten aller Symbole gleich groß sind, gilt:

$$p_i \cdot n_i = p_i \cdot \sum_j s_j \quad (4)$$

Daher kann der zweite Algorithmus sowohl für gleiche als auch für unterschiedliche Durchmesser benutzt werden.

2.3 Implementierung des Algorithmus

Der Baum ist über die Klasse *TreeNode* rekursiv implementiert. Jeder Knoten entspricht einem *TreeNode*-Objekt.

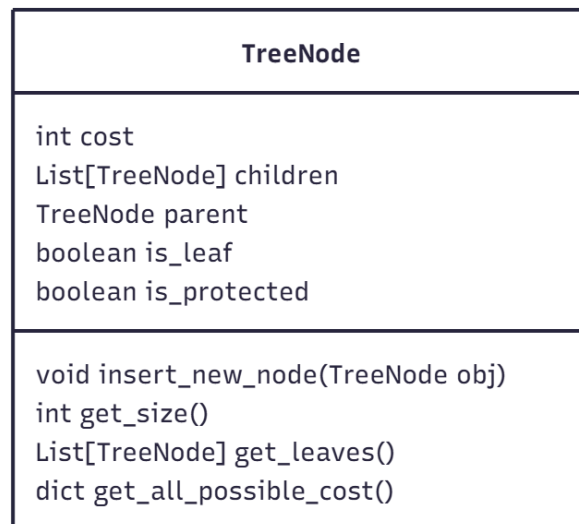


Abbildung 7: Klasse TreeNode

2.3.1 Erstellung des Baums

Zu Beginn wird ein balancierter Baum über die Methode *create_tree()* erstellt.

```
1 def create_tree(root: TreeNode) -> TreeNode:
```

Methode 1: *create_tree*

Die Methode *create_tree* bekommt über den Aufruf *get_all_possible_costs()* eine Liste von allen möglichen neuen Blättern, die der Baum als nächstes hinzufügen kann. Das Blatt mit den geringsten Kosten wird dem Baum über den Aufruf *insert_new_node(obj)* hinzugefügt. Falls der Elternknoten des neuen Blatts selbst noch ein Blatt ist, entsprechen die Kosten des neuen Blatts den kombinierten Kosten für die ersten beiden Blätter. Das liegt daran, dass jeder innere Knoten mindestens zwei Kinder haben muss – daher stellen diese Kosten die effektiv neu entstehenden Kosten für den Baum dar. Das geschieht solange, bis der Baum genauso viele Blätter hat, wie es Buchstaben gibt. Die Anzahl der Blättern kann mit der Methode *get_size()* ermittelt werden.

```
1 def get_all_possible_costs(self) -> dict:
```

Methode 2: *get_all_possible_costs*

Die Methode *get_all_possible_costs* prüft, ob das *TreeNode*-Objekt noch weitere Kinder aufnehmen kann und nicht protected ist. Falls dies der Fall ist, fügt es sich selbst sowie die Kosten des potenziellen neuen Kindes einem Dictionary hinzu. Anschließend wird die Methode rekursiv für alle bestehenden Kinder aufgerufen, so dass auch deren mögliche Erweiterungen im Dictionary erfasst werden.

```
1 def insert_new_node(self, obj: TreeNode):
```

Methode 3: `insert_new_node`

Die Methode `insert_new_node` prüft, ob das aktuelle Objekt selbst das übergebene `TreeNode`-Objekt ist. In diesem Fall fügt es ein neues Kind hinzu. Wenn vorher noch keine Kinder vorhanden sind, werden zwei Kinder hinzugefügt. Ansonsten wird die Methode rekursiv auf alle bestehenden Kinder angewandt.

2.3.2 Permutation des Baums

Sobald ein balancierter Baum erstellt wurde, muss dieser zum ausgearteten Baum umgewandelt werden. Dies geschieht über die Methode `find_best_tree`.

```
1 def find_best_tree(root: TreeNode) -> TreeNode:
```

Methode 4: `find_best_tree`

Die Methode `find_best_tree` versucht, den übergebenen Baum so lange zu permutiert, bis alle Blätter als *protected* markiert sind und keine Blätter mehr hinzugefügt werden können. Dazu wird der Iterationsschritt über den Aufruf `improve_tree(root)` wiederholt und mit der Methode `get_tree_cost(root)` werden jeweils die neuen Kosten des Baums berechnet. Wenn die Kosten ein neues Minimum sind, wird der aktuelle Baum als bester Zwischenstand gespeichert. Wenn dem Baum keine neuen Blätter hinzugefügt werden können, wird der bis dahin beste gefundene Baum zurückgegeben.

```
1 def improve_tree(root: TreeNode) -> TreeNode:
```

Methode 5: `improve_tree`

Zu Beginn holt sich die Methode `improve_tree` alle Blätter im Baum über den Aufruf `get_leaves()`. Diese Liste wird dann aufsteigend nach den Kosten jedes Blatts sortiert. Der Elternknoten des Blatts mit den höchsten Kosten wird daraufhin selbst zu einem Blatt und verliert alle Kinder. Um die fehlenden Blätter wieder hinzuzufügen, wird danach die Methode 1 (`create_tree`) aufgerufen.

```
1 def get_tree_cost(root: TreeNode) -> int:
```

Methode 6: `get_tree_cost`

Die Methode `get_tree_cost` berechnet die Gesamtkosten des Baums. Dafür holt sie sich auch über den Aufruf `get_leaves()` alle Blätter im Baum. Die Liste der Blätter wird dann nach den Kosten aufsteigend sortiert und jedes Element wird wie in der Gleichung (3) berechnet und zusammenaddiert.

2.4 Laufzeitanalyse und Einordnung des Algorithmus

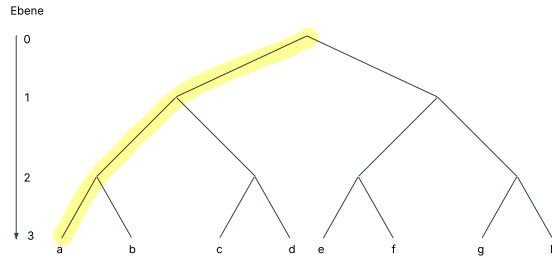


Abbildung 8: Balancierter Baum, Markierung der richtigen Knoten

2.4.1 Analyse des Algorithmus

Der Algorithmus endet, wenn keine Knoten mehr eingefügt werden können. Dies ist spätestens gegeben, wenn der Baum maximal ausgeartet ist (Abbildung 5). Allgemein kann jeder Knoten maximal $r-1$ mal verschoben werden, bis er komplett auf der linken Seite ist. Dabei ist r die Anzahl der verschiedenen Symbole, deren Kosten hierbei unerheblich sind. Daraus folgt, dass die maximale Anzahl an Iterationen bis zum maximal ausgearteten Baum

$$I \leq (r-1) \cdot \text{Anzahl falscher Knoten} \quad (5)$$

ist. Falsche Knoten sind dabei innere Knoten, die beim ausbalancierten Baum noch nicht auf der Stelle des ausgearteten Baums sind, also alle bis auf die äußersten linken Knoten (Abbildung 8).

Generell gilt, um die maximale Anzahl an inneren Knoten zu erhalten, sollte jeder Knoten möglichst wenige Kinder besitzen. Wie zuvor festgelegt, hat jeder innere Knoten mindestens zwei Kinder. Daraus folgt, dass die maximale Anzahl an inneren Knoten erreicht wird, wenn der Baum ein Binärbaum ist und durch

$$\text{innere Knoten} = w - 1 \quad (6)$$

berechnet werden kann.[7, Folie 5] Dabei ist w die Anzahl der Blättern. Auf jeder Ebene existiert ein richtiger innerer Knoten. Die Anzahl der Ebenen und somit der richtigen Knoten kann minimal mit

$$\text{richtige innere Knoten} = \log_r w \quad (7)$$

abgeschätzt werden.

Daher ergibt sich

$$I < (r-1) \cdot (w-1 - \log_r w) \quad (8)$$

Dies bedeutet eine Laufzeit von $O(r \cdot w)$.

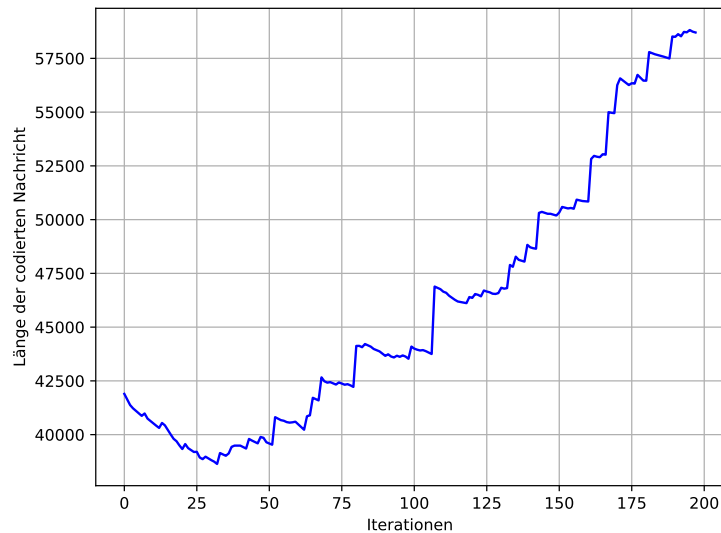


Abbildung 9: Länge der codierten Nachricht aufgetragen über mehrere Iterationen

In Abbildung 9 ist dargestellt, wie die Länge der codierten Nachricht des Testfalls *schmuck9.txt* [2] sich über die Iterationen hinweg verändert. Das Minimum wird bereits in der ersten Hälfte der Iterationen erreicht. Das zeigt gut, dass der Algorithmus kein Optimierungs-Algorithmus ist, sondern von einem Maximum zum anderen Maximum geht.

2.4.2 Stand der Forschung

Die vorliegende Fragestellung ist seit den 60er-Jahren des letzten Jahrhunderts in der theoretischen Informatik bekannt. Dabei lieferte Huffman eine Lösung für den Spezialfall von zwei Quellsymbolen.[8, S. 39]

Für unseren Fall von mehr als zwei Quellsymbolen mit unterschiedlichen Kosten entwickelte bereits Richard Karp 1961 eine Lösung.[6] Diese wurde 1998 von Mordecai J. Golin und Günter Rote verbessert und ist bis heute das schnellste Verfahren, um auf eine exakte Lösung zu kommen.[5]

”The minimum-cost prefix-free code for n words can be computed in $O(n^{C+2})$ time and $O(n^{C+1})$ space, if the costs of the letters are integers between 1 and C .”[5, S. 20]

Für die Beispielaufgabe *schmuck5.txt* [2] hat der Algorithmus eine Laufzeit von 34^8 ($n=34$; $C=6$) Iterationen. Diese Art von Algorithmus kann wegen seiner langen Laufzeit nicht für die vorgegebenen Beispielaufgaben verwendet werden.

Um eine polynomielle Laufzeit zu erreichen, entwickelten Mordecai J. Golin, Claire Mathieu und Neal E. Young im Jahre 2012 ein Approximationsverfahren.

Dieser Algorithmus basiert auf dem PTAS-Verfahren (Polynomial-time Approximation Scheme), wobei zuerst ein präfixfreier Code bis zu einer bestimmten Tiefe τ erstellt wird und dann die restlichen Codewörter durch ein Rundungsverfahren ergänzt werden. Dabei ist es zentral, dass die Wahrscheinlichkeitsverteilung der zu kodierenden Codewörter möglichst in großen Gruppen mit gleich großer Wahrscheinlichkeit eingeordnet werden

kann.[4] Im Testfall *schmuck9.txt* [2] ist dies zum Beispiel nicht gegeben, weshalb das Bestimmen des Vorbaums mehr als 9^{24} ($\varepsilon=0,45$; $\tau=9$) Schritte benötigt. Somit ist dieses Verfahren für die gewählten Beispiele ebenfalls nicht verwendbar.

2.4.3 Fazit

Für die gegebene Fragestellung gibt es bislang weder einen Beweis, dass sie NP-schwer ist, noch einen Gegenbeweis, dass sie es nicht ist. Aktuell bekannte Verfahren für exakte Ergebnisse besitzen eine exponentielle Laufzeit.[5] Mein gefundener Algorithmus findet nicht die beste Lösung für das gegebene Problem. Er liefert jedoch in polynomieller Laufzeit eine gute Approximation und ist daher eine gute Abwägung zwischen Laufzeit und Genauigkeit. Somit wird nicht beantwortet, ob die Fragestellung NP-schwer ist.

2.5 Vergleich mit der Musterlösung

In der Musterlösung werden drei Lösungen vorgeschlagen. Der erste Ansatz ist ein Greedy-Algorithmus basierend auf den Huffman Algorithmus. Die zweite Variante ist ein ILP-Ansatz nach Richard Karp und die dritte ist dessen Erweiterung von Mordecai J. Golin und Günter Rote.[3]

Alle drei Ansätze waren mir bei Abgabe meiner Arbeit bekannt. Trotzdem entschied ich mich dagegen, eines dieser Verfahren zu implementieren, vor allem aufgrund deren hoher Laufzeit, und entwickelte stattdessen mein eigenes Verfahren. Mein Algorithmus hat dabei für die Testfälle eine durchschnittliche Abweichung von 1,27% vom Idealergebnis, besitzt jedoch im Gegensatz zu den Algorithmen in der Musterlösung eine sehr viel kürzere Laufzeit [3] (siehe auch 5.1.1). Für mein gutes Ergebnis erhielt ich daher einen Pluspunkt und erreichte insgesamt 21 Punkte für diese Aufgabe.

3 Aufgabe 2: Simultane Labyrinth

3.1 Problemstellung

In dieser Aufgabe möchten drei Freunde, Anton, Bea und Chris, zwei Labyrinth gleichzeitig durchlaufen, die dieselben rechteckigen Maße (n, m) haben. Zwei Personen starten jeweils in ihrem Labyrinth bei der Position $(0, 0)$ und müssen das Ziel (n, m) erreichen. Die dritte Person gibt beiden jeweils immer dieselbe nächste Schrittanweisung. Wenn der nächste Schritt in einem Labyrinth durch eine Wand blockiert ist, bleibt die Person stehen. Gesucht ist eine möglichst kurze Abfolge von Schritten, die beide Labyrinth mit denselben Befehlen löst. Die ursprüngliche Aufgabe enthält Labyrinth ohne Gruben. In der zweiten Teilaufgabe werden die Labyrinth um diese erweitert. Wenn ein Spieler in eine Grube fällt, wird seine Position auf die Startposition $(0, 0)$ zurückgesetzt. [1]

3.2 Theoretische Erklärung des Algorithmus

3.2.1 Erstellung der Matrix

Zu Beginn wird für jedes Labyrinth eine Matrix erstellt (Abbildung 10), in der für jede Position ein Tupel aus einem Kostenwert und einer Richtung gespeichert wird. Die gespeicherte Richtung zeigt den optimalen Folgeschritt zum Ziel und ist in Tabelle 1 dargestellt.

Code	Richtung
0	Rechts
1	Links
2	Oben
3	Unten

Tabelle 1: Codierung der Richtungen im Labyrinth

Der Kostenwert cp gibt an, wie viele Schritte bis zum Ziel benötigt werden. In Abbildung 10 ist ein Beispiel Labyrinth mit dazugehöriger Matrix abgebildet.

13;3	14;1	5;0	4;0	3;3
12;3	13;1	14;1	5;2	2;3
11;3	8;0	7;0	6;2	1;3
10;0	9;2	8;2	7;2	0;null

Abbildung 10: Labyrinth mit dazugehöriger Matrix

Zum Erstellen durchläuft der Algorithmus rückwärts ausgehend vom Endpunkt alle erreichbaren Nachbarpositionen. Für jeden Nachbarn wird in der Matrix ein Kostenwert eingetragen, der um eins höher ist als der Kostenwert des aktuellen Punkts, sowie die Richtung, aus der der Nachbar erreicht wurde.

3.2.2 Lösen der Labyrinth

Der Algorithmus basiert auf einer Liste der aktuellen besten Wege. In jedem Iterationsschritt wird der momentan beste Weg um einen Schritt erweitert, indem in den Matrizen geprüft wird, welchen Schritt die beiden Labyrinth für ihre jeweilige Position vorschlagen. Dieser Schritt wird an die bisherige Schrittfolge angehängt. Dabei entstehen in der Regel zwei unterschiedliche Wege, die zur Liste der bisherigen Wege hinzugefügt werden, sofern nicht beide Labyrinth denselben Schritt vorschlagen. Die Auswertung der bisherigen Wege erfolgt mithilfe dieser Rechnung

$$c_b = cp_{s1} + cp_{s2} \quad (9)$$

$$c_a = cp_{a1} + cp_{a2} \quad (10)$$

$$c_w = \frac{c_b - c_a}{n} \quad (11)$$

wobei gilt:

- $s1$ und $s2$ sind die Startposition

- $a1$ und $a2$ sind die aktuellen Positionen

- c_w sind die Kosten des bisherigen Wegs

- n ist die Anzahl der benötigten Schritte

Der Bruch c_w beschreibt die durchschnittliche Einsparung von Kosten pro Schritt. Je höher der Wert ist, desto effizienter ist der bisherige Pfad. Dabei ist 2 der maximal erreichbare Wert. Dieser ist nur bei identischen Labyrinth gegeben.

Der Weg mit der höchsten Einsparung gilt dabei als momentan bester Weg. Nach jeder Iteration wird das aktuelle Positionspaar $((x_1, y_1), (x_2, y_2))$ gespeichert, um zu verhindern, dass der Algorithmus exakt dasselbe Positionspaar noch einmal besucht. Zusätzlich gibt es auch die Begrenzung l , die festlegt, wie viele unterschiedliche Wege, sortiert nach ihren Kosten, in der Liste gespeichert werden dürfen. Diese Begrenzung beeinflusst maßgeblich die Laufzeit und die Genauigkeit des Algorithmus. In der Liste dürfen die Wege unterschiedliche Werte für n haben. Denn wichtig ist vor allem, dass ihre Kosten (Gleichung (11)) gering sind. Der gesamte Ablauf wird solange wiederholt, bis beide im Ziel sind.

Bei der Erweiterung des Algorithmus werden Gruben sowohl in der Kostenmatrix als auch beim Lösen wie normale Felder gesehen. Wenn eine der beiden Personen eine Grube berührt, wird ihre Position auf die jeweilige Startposition zurückgesetzt. Daraus folgt, dass die Kosten der Grubenfelder den Kosten der Startpositionen entsprechen.

3.3 Implementierung des Algorithmus

3.3.1 Erstellung der Matrix

Zur Erstellung der Matrix wird die Klasse *Cost_Matrix_Generator* benutzt.

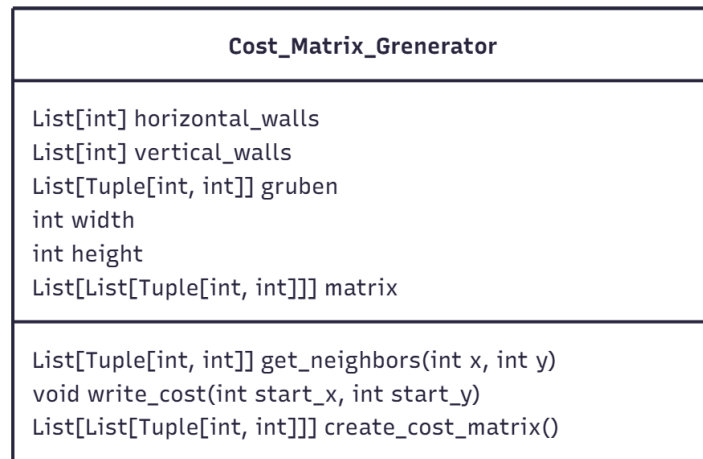


Abbildung 11: Klasse Cost_Matrix_Generator

Für jedes Labyrinth wird ein *Cost_Matrix_Generator*-Objekt erstellt und die Methode *cost_obj.create_cost_matrix()* aufgerufen. Das *Cost_Matrix_Generator*-Objekt bekommt als Übergabewerte die horizontalen bzw. vertikalen Wände sowie die Gruben und die Größe des Labyrinths.

```
1 def create_cost_matrix(self) -> list:
```

Methode 7: *create_cost_matrix*

Die *create_cost_matrix* Methode erstellt zunächst eine leere Liste mit der Dimension [height][width], die als Kostenmatrix dient. Dann ruft sie die Methode *write_cost* auf, um die Kostenmatrix zu füllen.

```
1 def write_cost(self, start_x: int, start_y: int):
```

Methode 8: *write_cost*

Zu Beginn der Methode *write_cost()* wird die Liste *stack* initialisiert. Sie speichert alle Felder, die noch besucht werden müssen. Jeder Eintrag in der Liste besitzt vier Werte, das heißt die x-Position, die y-Position, die aktuellen Kosten sowie die entgegengesetzte Richtung, aus welcher das Feld betreten wurde. Die Endposition wird als erstes Element hinzugefügt. Anschließend wird das erste Element der Liste entnommen und in die Kostenmatrix übertragen. Daraufhin werden alle benachbarten Felder mit der Methode *get_neighbours* ermittelt und dem *stack* hinzugefügt. Dabei werden die Kosten um eins erhöht und die jeweilige Richtung gespeichert. Das wird solange wiederholt, bis der *stack* leer ist. Falls ein Element schon in der Kostenmatrix ist, wird es übersprungen.

3.3.2 Lösen der Labyrinth

Zum Lösen der Labyrinth wird die Klasse *Solving* benutzt.

Solving
<pre> int width int height int cost_at_the_beginning List[Tuple[int, int]] gruben_1 List[Tuple[int, int]] gruben_2 List[int] horizontal_walls_1 List[int] vertical_walls_1 List[int] horizontal_walls_2 List[int] vertical_walls_2 List[List[Tuple[int, int]]] cost_matrix_1 List[List[Tuple[int, int]]] cost_matrix_2 dict visited </pre>
<pre> Tuple[int, int] next_position(Tuple[int, int] position, int next_move) int next_move(Tuple[int, int] position, List[List[Tuple[int, int]]] cost_matrix) int get_moving_cost(Tuple[int, int] position, List[List[Tuple[int, int]]] cost_matrix) int get_total_cost(Tuple[Tuple[int, int], Tuple[int, int]] positions) Move check_visited(Tuple[Tuple[int, int], Tuple[int, int]] next_position) List[Move] neighbours_cost(Move move) </pre>

Abbildung 12: Solving Klasse

Die Wege werden mit der Klasse *Move* dargestellt.

Move
<pre> List[int] moves Tuple[Tuple[int, int], Tuple[int, int]] positions int cost float weight </pre>

Abbildung 13: Move Klasse

Das Attribut *cost* beschreibt die aufsummierten Kosten der beiden aktuellen Positionen basierend auf den Werten aus den Kostmatrizen. Das Attribut *weight* (entspricht c_w) wird gemäß der Gleichung (11) berechnet.

Zu Beginn wird ein *Solving*-Objekt initialisiert und eine leere Liste *moves_stack* angelegt, in der alle aktuellen Wege gespeichert werden. Anschließend wird ein erstes *Move*-Objekt erzeugt, dessen Startposition $((0, 0), (0, 0))$ ist und dessen moves leer sind. In jedem Schritt wird das erste Element aus der Liste *moves_stack* entnommen. Dann wird mit folgendem Aufruf

```
1 new_moves = solving_obj.neighbours_cost(move)
```

eine Liste von möglichen Folgewegen erzeugt. Die Liste wird mit den neuen Wegen erweitert, sortiert und auf die l besten Wege beschränkt.

```
1 def neighbours_cost(self, move: Move) -> list:
```

Methode 9: `neighbours_cost`

Zu Beginn der Methode `neighbours_cost()` werden die beiden Positionen aus dem Attribut `positions` des übergebenen `move`-Objekts genommen. Anschließend werden mit den folgenden Aufrufen

```
1 move_1 = self.next_move(pos1, self.cost_matrix_1)
  move_2 = self.next_move(pos2, self.cost_matrix_2)
```

die jeweils nächsten möglichen Richtungen aus der Kostenmatrix bestimmt. Daraufhin werden die neuen zwei Positionspaare berechnet und es wird überprüft, ob die beiden Positionspaare bereits besucht wurden.

Zum Schluss wird für jeden möglichen Zug ein `Move`-Objekt erstellt und sie werden zusammen in einer Liste übergeben. Falls eines der neuen Positionspaare bereits besucht wurde oder bereits im Ziel ist, wird der entsprechende Zug nicht übergeben. Der gesamte Ablauf von `neighbours_cost` ist in Abbildung 17 dargestellt.

3.4 Laufzeitanalyse

Von jedem Positionspaar aus existieren im schlechtesten Fall zwei mögliche Schritte, entweder in Richtung des Ziels von Labyrinth 1 oder in Richtung des Ziels von Labyrinth 2. Das führt zu maximal $2^{(nm)^2}$ verschiedenen Möglichkeiten an Schritten. Aufgrund dieser großen Anzahl an Möglichkeiten untersucht der Algorithmus nicht alle Pfade vollständig, sondern macht nur eine Abschätzung. Der hier vorgestellte Ansatz garantiert daher nicht unbedingt den optimalen Lösungsweg, liefert jedoch in akzeptabler Laufzeit eine gute Annäherung. Der Algorithmus ähnelt einer Tiefensuche, da er einige Wege tief verfolgt, während andere nur kurz betrachtet werden.

Wie oben erwähnt beeinflusst die Begrenzung l die Laufzeit und die Genauigkeit des Algorithmus stark.

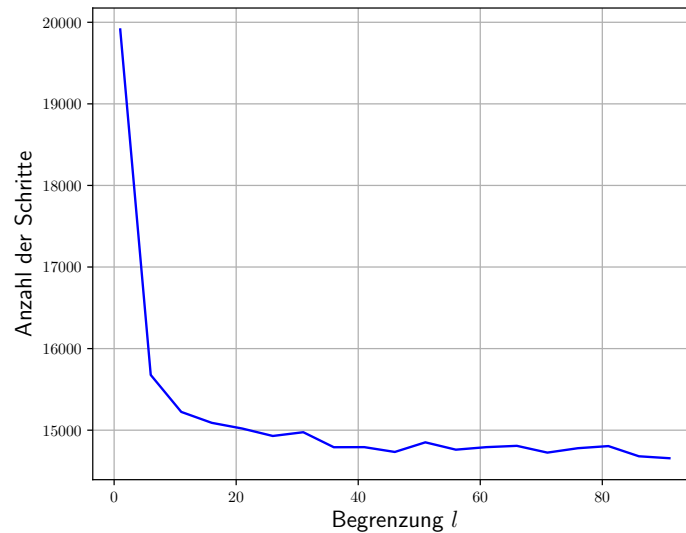


Abbildung 14: Anzahl der Schritte in Abhängigkeit von der Begrenzung l

Abbildung 14 zeigt die Anzahl der Schritte des gefundenen Lösungswegs in Abhängigkeit vom Parameter l für Testfall *labyrinth4.txt* [2].

Auffällig ist, dass der Graph mit zunehmendem l gegen die Anzahl der Schritte des optimalen Wegs konvergiert. Ebenso erkennbar ist, dass der Graph nicht streng monoton fallend ist. Wenn die Steigung positiv ist, wird für einen größeren Wert von l ein schlechterer Pfad gefunden. Dies liegt daran, dass durch den höheren Wert von l neue Wege betrachtet werden. Diese erscheinen kurzfristig vorteilhafter als die bisherigen Wege und verdrängen dadurch die eigentlich besseren aus der Liste. Das führt insgesamt zu einem schlechteren Weg. Bei größeren l -Werten tritt dieser Effekt jedoch kaum noch auf, da die Liste groß genug ist, um alle relevanten Pfade zu behalten. Der Einfluss dieser Monotonieveränderungen wird daher mit wachsendem l vernachlässigbar.

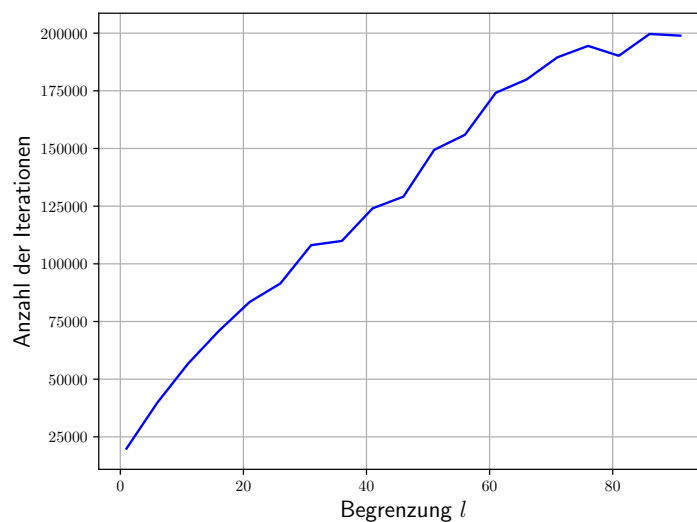


Abbildung 15: Anzahl der Iterationen in Abhängigkeit von der Begrenzung l

Abbildung 15 stellt die Anzahl an Iterationen dar, die zur Bestimmung des Lösungswegs benötigt werden in Abhängigkeit von Parameter l . Der Graph steigt mit wachsendem l stark an.

Allgemein kann die maximale Anzahl an Iterationen in Abhängigkeit von l wie folgt berechnet werden:

$$l \cdot (2c_1 + c_2) \quad (12)$$

wobei gilt:

- c_1 sind die Kosten vom Startpunkt aus der Kostenmatrix 1
- c_2 sind die Kosten vom Startpunkt aus der Kostenmatrix 2

Der Faktor $2c_1 + c_2$ beschreibt die maximale Anzahl an Schritten, aus denen der schlechteste Lösungsweg besteht. Zunächst erreicht Person 1 das Ziel mit c_1 Schritten. Anschließend läuft Person 2 denselben Weg zurück in ebenfalls c_1 Schritten und geht dann selber ins Ziel mit c_2 Schritten. Im schlechtesten Fall geht sie für jeden Eintrag in der Liste, also l mal, die maximale Anzahl an Schritten. Daraus resultiert die Gleichung (12). Dies bedeutet, dass sich eine Laufzeit von der Ordnung $o(l)$ ergibt. Praktisch gesehen ist die Laufzeit aber sehr viel kleiner als die maximale Laufzeit (Abbildung 16).

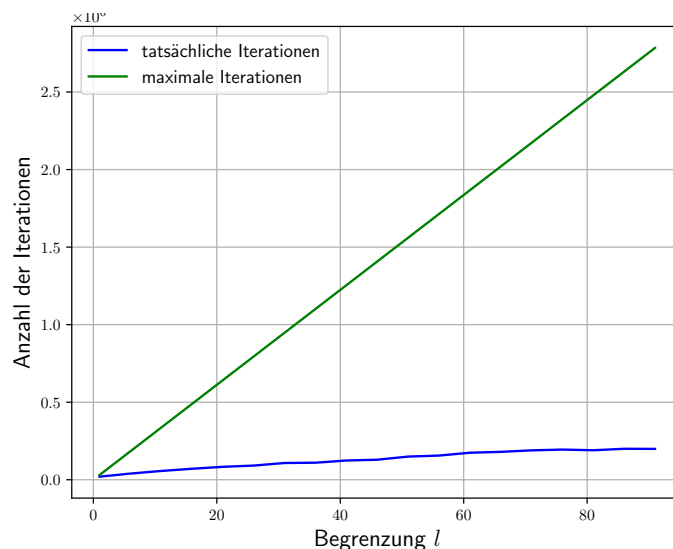


Abbildung 16: Maximale Iterationen gegenüber tatsächlichen Iterationen aufgetragen gegenüber der Begrenzung l

Die Anzahl der Iterationen steht in einem linearen Zusammenhang mit den Werten von l , während die Anzahl der Schritte bei größeren l -Werten gegen das optimale Ergebnis konvergiert. Je größer der Wert von l , desto mehr Iterationen werden durchgeführt und desto besser wird das Ergebnis. Die Genauigkeit lässt sich anhand des Parameters l jedoch nicht allgemein bestimmen, da sie stark von der Struktur des jeweiligen Labyrinths abhängt.

3.5 Vergleich mit der Musterlösung

In der Musterlösung werden zwei mögliche Lösungsstrategien vorgestellt. Die erste Strategie besteht in der Anwendung der Breitensuche auf zwei Labyrinth. Diese Variante wird jedoch in der Musterlösung aufgrund ihrer hohen Laufzeit als nicht praktikabel eingestuft. Die zweite Strategie basiert auf dem A-Star Algorithmus mit Priority Queue.[3] Diese Vorgehensweise ähnelt stark meiner eigenen Idee. Wie in meiner Kostenmatrix wird auch hier für jede Position die Distanz zum Ziel berechnet. Der wesentliche Unterschied liegt vor allem in dem effizienteren Warteschlangenmodell der Musterlösung.[3]

Insgesamt lagen meine Ergebnisse, abgesehen von Testfall *labyrinth8.txt* [2], bei 1,1% Abweichung vom Optimalwert.[3] (siehe auch 5.1.2) Bei Testfall *labyrinth8.txt* fiel mein Ergebnis etwa 3-mal so groß aus, weshalb mir 2 Punkte im Kriterium „Verfahren mit guten Ergebnissen“ abgezogen wurden. Außerdem war meine Überlegung zur Laufzeit des Verfahrens bei Abgabe des Wettbewerbs nicht ausreichend detailliert, was zu einem weiteren Punktabzug führte.

Für die Aufgabe erhielt ich 17 von 20 Punkten.

4 Fazit

Insgesamt konnte ich mit 38 von 40 Punkten den zweiten Platz erreichen und gehörte damit zu den 60 besten Teilnehmenden von über 950 zugelassenen Teilnehmern. Aufgrund dieser Leistung wurde ich für die Internationale Informatik-Olympiade vorgeschlagen und nehme derzeit an deren Trainingsprogramm teil. Durch diese Arbeit sammelte ich erste Erfahrungen in der Laufzeitanalyse eigener Algorithmen und im Umgang mit aktuellen wissenschaftlichen Texten. Aus meiner Erfahrung erweist sich der Wettbewerb insgesamt als gute Möglichkeit, Schüler an wissenschaftliches Arbeiten und algorithmisches Denken heranzuführen.

5 Anhang

Im Folgenden ist die Methode *neighbours_cost* als Flussdiagramm dargestellt.

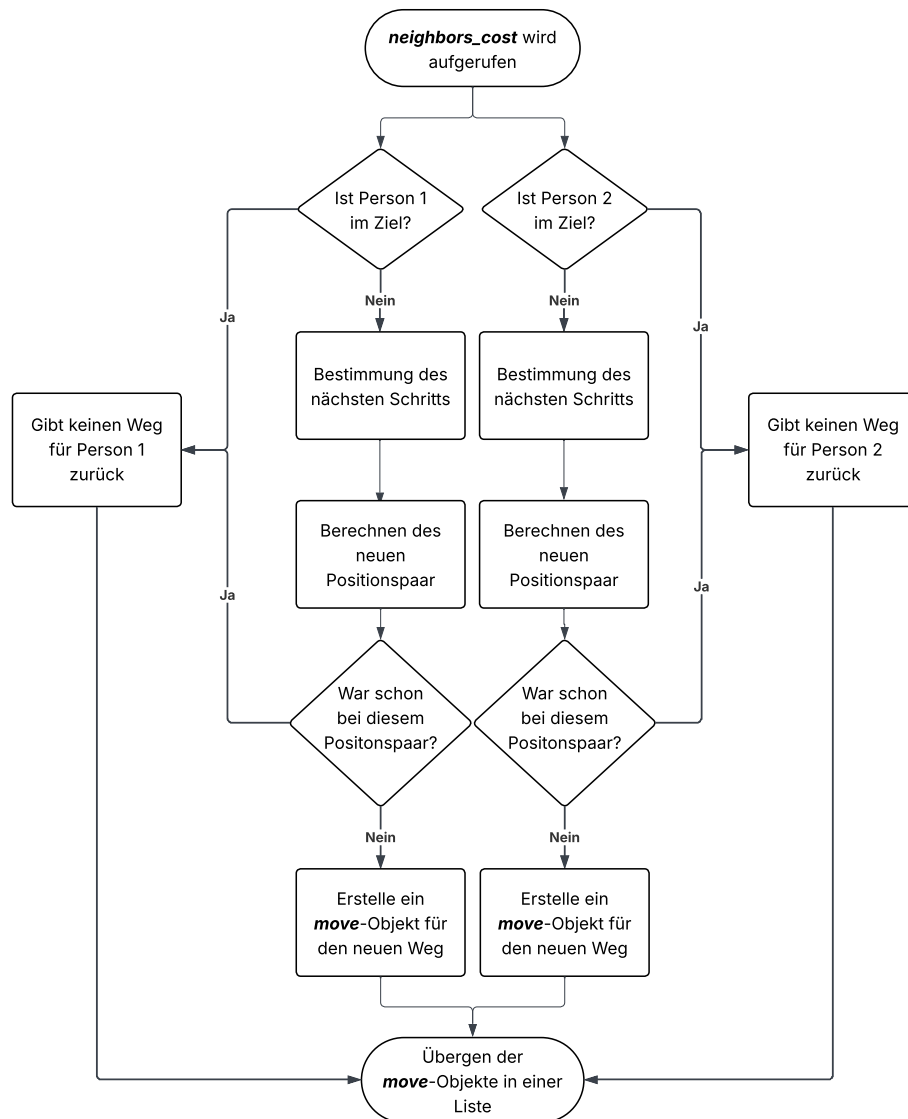


Abbildung 17: *neighbours_cost* Flussdiagramm

5.1 Lösungen der Testfälle

Die folgenden Ergebnisse zeigen meine Resultate der durchgeführten Testfälle im Vergleich zur Musterlösung. Die Testfälle stammen aus der Quelle der [2], die optimalen Ergebnisse aus der Quelle der [3].

5.1.1 Aufgabe 1: Schmucknachrichten

Die Lösungen können in Tabelle 2 eingesehen werden.

Testfall	meine Gesamtkosten	optimales Ergebnis
schmuck0.txt	114	113
schmuck00.txt	382	372
schmuck01.txt	1165	1150
schmuck1.txt	194	191
schmuck2.txt	135	135
schmuck3.txt	279	279
schmuck4.txt	137	137
schmuck5.txt	3189	3162
schmuck6.txt	234	234
schmuck7.txt	135409	134559
schmuck8.txt	3358	3287
schmuck9.txt	39635	36597

Tabelle 2: Lösungen der Aufgabe 1

5.1.2 Aufgabe 2: Simultane Labyrinth

Die Lösungen können in Tabelle 3 eingesehen werden.

Testfall	meine Gesamtkosten	optimales Ergebnis
labyrinth0.txt	8	8
labyrinth1.txt	31	31
labyrinth2.txt	66	65
labyrinth3.txt	166	164
labyrinth4.txt	14497	14384
labyrinth5.txt	1345	1308
labyrinth6.txt	1844	1844
labyrinth7.txt	keine Lösung	keine Lösung
labyrinth8.txt	1433	472
labyrinth9.txt	1036	1012

Tabelle 3: Lösungen der Aufgabe 2

5.2 Quellcode

5.2.1 Aufgabe 1: Schmucknachrichten (gekürzt)

```

class TreeNode:
2
    def __init__(self, cost, parent, perl_code):
4        self.perl_code = perl_code # Speichert die Codierung für den Knoten
        self.cost = cost
6        self.children = []
        self.parent = parent
8        self.is_leaf = True
        self.is_protected = False # Wenn True, dann kann er keine kinder
        mehr bekommen

```



```

10
def insert_new_node(self, obj):
12     if obj is self:
13         if self.is_leaf: # Damit jede parent immer mehr mindestens zwei
14             kinder hat
15                 self.is_leaf = False
16                 treenode = TreeNode(self.cost + perl_costs[0], self, self.
17 perl_code+[len(self.children)])
18                 self.children.append(treenode)
19
20                 treenode = TreeNode(self.cost + perl_costs[len(self.children)],
21 self, self.perl_code+[len(self.children)])
22                 self.children.append()
23                 return
24
25     for child in self.children:
26         child.insert_new_node(obj)
27
28 def get_size(self) -> int:
29     if self.is_leaf:
30         return 1
31     return sum(child.get_size() for child in self.children)
32
33 def get_leaves(self) -> list:
34     if self.is_leaf:
35         return [self]
36
37     children_leaves = []
38     for child in self.children:
39         for i in child.get_leaves():
40             children_leaves.append(i)
41     return children_leaves
42
43 def get_all_possible_costs(self) -> dict:
44     children_cost = {}
45     if len(self.children) < len(perl_costs) and !self.is_protected:
46         if self.is_leaf:
47             children_cost[self] = self.cost + perl_costs[0] + self.cost +
48 perl_costs[1]
49         else:
50             children_cost[self] = self.cost + perl_costs[len(self.
51 children)]
52
53     for child in self.children:
54         children_cost.update(child.get_all_possible_costs())
55     return children_cost
56
57 def get_internal_nodes(self):
58     if self.is_leaf:
59         return 0

```

```

        return sum(child.get_internal_nodes() for child in self.children) + 1

1 def create_tree(root: TreeNode) -> TreeNode:
    while root.get_size() < len(distribution):
3         possible_children = root.get_all_possible_costs()
            smallest_value = min(possible_children.items(), key=lambda item: item
                [1])
5
            root.insert_new_node(smallest_value[0])
7     return root

9
def get_tree_cost(root):
11     all_leaves = root.get_leaves()
        all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.
            children)))
13     total_count = 0
        for i in range(len(all_leaves)):
15         total_count += all_leaves[i].cost * distribution[i]
        return total_count
17

19 def improve_tree(root: TreeNode) -> TreeNode:
    all_leaves = root.get_leaves()
21     all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.
        children)))
        highest_cost = 0
23     parent_node = None
        for i in range(len(all_leaves)):
25         if all_leaves[i].parent.parent is not None:
            if all_leaves[i].cost * distribution[i] > highest_cost:
27                 highest_cost = all_leaves[i].cost * distribution[i]
                    parent_node = all_leaves[i].parent
29
        parent_node.is_leaf = True
31     parent_node.children = []
        parent_node.is_protected = True
33
        return create_tree(root)
35

37 def find_best_tree(root):
    best_value = get_tree_cost(root)
39     best_root = copy.deepcopy(root)
        while True:
41         try:
            root = improve_tree(root)
43         new_calc = get_tree_cost(root)

```

```

45     if new_calc < best_value:
46         best_value = new_calc
47         best_root = copy.deepcopy(root)
48     except Exception as e:
49         # Gibt keine Verbesserung mehr
50         return best_root
51
52
53 def output(root):
54     print(f"Gesamtkosten: {get_tree_cost(root)}")
55     all_leaves = root.get_leaves()
56     all_leaves = sorted(all_leaves, key=lambda x: (x.cost, len(x.parent.
57         children)))
58     for i in range(len(all_leaves)):
59         print(f"{characters[i]}: {all_leaves[i].perl_code}")
60
61 def main():
62     # Initiiere Wurzel objekt
63     root = TreeNode(0, None, [])
64
65     # Erstellen des balancierten Baums
66     root = create_tree(root)
67
68     # Verbessern des Baums
69     root = find_best_tree(root)
70     output(root)

```

5.2.2 Aufgabe 2: Simultane Labyrinth (gekürzt)

```

1 class Cost_Matrix_Generator:
2     def write_cost(self, start_x, start_y):
3         stack = [(start_x, start_y, 0, None)]
4
5         while stack:
6             x, y, cost, direction = stack.pop()
7             if self.matrix[y][x] != 0:
8                 continue
9
10            self.matrix[y][x] = (cost, direction)
11
12
13            a = self.get_neighbours(x, y)
14            for neighbour in a:
15                nx, ny, ndirection = neighbour
16                if self.matrix[ny][nx] == 0:
17                    stack.append((nx, ny, cost + 1, ndirection))

```

```

19
    def create_cost_matrix(self):
21         self.matrix = [[0 for _ in range(self.width)] for _ in range(self.
height)]
        self.write_cost(self.width - 1, self.height - 1)
23         return self.matrix

1 class Move:
    def __init__(self, moves, positions, cost, heights_cost, weight=None):
3         self.moves = moves
        self.positions = positions
5         self.cost = cost
        self.weight = weight if weight is not None else (heights_cost - cost) /
len(moves) # moves are sorted based on this value

1 class Solving:
    def __init__(self, cost_matrix_1, cost_matrix_2, vertical_walls_1,
horizontal_walls_1,
3         vertical_walls_2, horizontal_walls_2, gruben_1, gruben_2,
width, height):
        self.width = width
5         self.height = height
        self.cost_at_beginning = cost_matrix_1[0][0][0] + cost_matrix_2
[0][0][0]
7
        self.gruben_1 = gruben_1
9         self.gruben_2 = gruben_2

11         self.vertical_walls_1 = vertical_walls_1
        self.horizontal_walls_1 = horizontal_walls_1
13
        self.vertical_walls_2 = vertical_walls_2
15         self.horizontal_walls_2 = horizontal_walls_2

17         self.cost_matrix_1 = cost_matrix_1
        self.cost_matrix_2 = cost_matrix_2
19
        self.visited = {}
21         # Funktionen und Bewegungsdeltas nur einmal definieren
        self.move_funcs = [test_right, test_left, test_up, test_down]
23         self.move_deltas = [(1, 0), (-1, 0), (0, -1), (0, 1)]

25 def next_position(self, position, next_move):
        mf = self.move_funcs
27         md = self.move_deltas
        pos1 = position[0]
29         pos2 = position[1]

```

```

31     # Für die erste Position
    if pos1 != (self.width - 1, self.height - 1):
33         matrix1 = self.vertical_walls_1 if next_move < 2 else self.
horizontal_walls_1
        if mf[next_move](pos1[0], pos1[1], matrix1):
35             pos1 = (pos1[0] + md[next_move][0], pos1[1] + md[next_move][1])
            if pos1 in self.gruben_1:
37                 pos1 = (0, 0)

39     # Für die zweite Position
    if pos2 != (self.width - 1, self.height - 1):
41         matrix2 = self.vertical_walls_2 if next_move < 2 else self.
horizontal_walls_2
        if mf[next_move](pos2[0], pos2[1], matrix2):
43             pos2 = (pos2[0] + md[next_move][0], pos2[1] + md[next_move][1])
            if pos2 in self.gruben_2:
45                 pos2 = (0, 0)
    return pos1, pos2

47 def next_move(self, pos, cost_matrix):
49     w = self.width
    h = self.height
51     return None if pos == (w - 1, h - 1) else cost_matrix[pos[1]][pos
[0]][1]

53 def get_moving_cost(self, pos, cost_matrix):
    w = self.width
55     h = self.height
    return 0 if pos == (w - 1, h - 1) else cost_matrix[pos[1]][pos[0]][0]
57
def get_total_cost(self, positions):
59     pos1, pos2 = positions
    return self.get_moving_cost(pos1, self.cost_matrix_1) + self.
get_moving_cost(pos2, self.cost_matrix_2)
61
def check_visited(self, next_position, length, move):
63     # Erstelle einen Schlüssel als Tupel, das beide Positionen enthält
    key = (next_position[0][0], next_position[0][1],
65           next_position[1][0], next_position[1][1])
    if key in self.visited and self.visited[key] <= length:
67         return None
    self.visited[key] = length
69     return move

71 def neighbours_cost(self, move: Move):
    # Berechne beide Positionen anhand der Bewegungsfolge
73     pos1 = move.positions[0]
    pos2 = move.positions[1]
75
    # Ermittle den nächsten Zug für beide Positionen
77     move_1 = self.next_move(pos1, self.cost_matrix_1)

```

```

    move_2 = self.next_move(pos2, self.cost_matrix_2)

79
    # Nächsten Positionspaare berechnen
81    next_position_1 = self.next_position(move.positions, move_1) if move_1
    is not None else None
    next_position_2 = self.next_position(move.positions, move_2) if move_2
    is not None else None

83
    # Testen ob die Positionspaare schon einmal da waren
85    move_1 = self.check_visited(next_position_1, len(move.moves) + 1,
    move_1) if move_1 is not None else None
    move_2 = self.check_visited(next_position_2, len(move.moves) + 1,
    move_2) if move_2 is not None else None

87
    results = []
89    if move_1 is not None:
        results.append(Move(move.moves + [move_1], next_position_1, self.
        get_total_cost(next_position_1), self.cost_at_beginning))
91    if move_2 is not None:
        results.append(Move(move.moves + [move_2], next_position_2, self.
        get_total_cost(next_position_2), self.cost_at_beginning))
93    return results


1 def create_matrix(data, height):
    vertical_walls = [data.pop(0) for _ in range(height)]
    3 horizontal_walls = [data.pop(0) for _ in range(height - 1)]
    gruben_anzahl = int(data.pop(0)[0])
    5 gruben = [tuple(data.pop(0)) for _ in range(gruben_anzahl)]
    return horizontal_walls, vertical_walls, gruben

7

9 def create_matrix_for_laby(horizontal_walls, vertical_walls, gruben, width,
    height):
    cost_obj = Cost_Matrix_Generator(horizontal_walls, vertical_walls, gruben
    , width, height)
11 return cost_obj.create_cost_matrix()

13
def moving(cost_matrix_1, cost_matrix_2, width, height, horizontal_walls_1,
    vertical_walls_1, horizontal_walls_2, vertical_walls_2, gruben_1,
    gruben_2, 1):
15

17 solving_obj = Solving(cost_matrix_1, cost_matrix_2, vertical_walls_1,
    horizontal_walls_1, vertical_walls_2, horizontal_walls_2, gruben_1,
    gruben_2, width, height)

19 moves = [Move([], ((0, 0), (0, 0)), 0, heights_cost=0, weight=0)]

```

```

21 while True:
22     # Erstes Element aus der Liste nehmen
23     move = moves[0]
24     moves.pop(0)
25
26     # Liste mit den neuen Wegen erweitern
27     new_moves = solving_obj.neighbours_cost(move)
28     moves.extend(new_moves)
29
30     moves = sorted(moves, key=lambda obj: (-obj.weight, len(obj.moves)))
31     moves = moves[:1] # Nur die 1 besten Züge speichern
32
33     if moves[0].cost == 0:
34         return moves[0]
35
36
37 def output(move):
38     print(len(move.moves))
39     print(move.moves)
40
41
42 def main():
43     # Datei Einlesen
44     width, height, data = read_data_from_file('input/labyrinthe8.txt')
45
46     # Listen erstellen
47     horizontal_walls_1, vertical_walls_1, gruben_1 = create_matrix(data,
48         height)
49     horizontal_walls_2, vertical_walls_2, gruben_2 = create_matrix(data,
50         height)
51
52     # Matrix für beide Labyrinthe erstellen
53     cost_matrix_1 = create_matrix_for_laby(horizontal_walls_1,
54         vertical_walls_1, gruben_1, width, height)
55     cost_matrix_2 = create_matrix_for_laby(horizontal_walls_2,
56         vertical_walls_2, gruben_2, width, height)
57
58     # Weg finden
59
60     l = 100
61     try:
62         move = moving(cost_matrix_1, cost_matrix_2, width, height,
63             horizontal_walls_1, vertical_walls_1, horizontal_walls_2,
64             vertical_walls_2, gruben_1, gruben_2, l)
65         output(move)
66
67     except:
68         print("No solution found")
69

```

```
65 if __name__ == "__main__":  
    main()
```

5.3 Quellen

- [1] Aufgerufen am 11.10.2025. URL: https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/43/2_runde/Aufgaben432.pdf.
- [2] Aufgerufen am 12.10.2025. URL: <https://bwinf.de/bundeswettbewerb/43/#c4522>.
- [3] Aufgerufen am 11.10.2025. URL: https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/43/2_runde/Loesungshinweise432.pdf.
- [4] Mordecai J. Golin, Claire Mathieu und Neal E. Young. „Huffman Coding with Letter Costs: A Linear-Time Approximation Scheme“. In: *SIAM Journal on Computing* 41.3 (2012), S. 684–713.
- [5] Mordecai J. Golin und Günther Rote. „A Dynamic Programming Algorithm for Constructing Optimal Prefix-Free Codes with Unequal Letter Costs“. In: *IEEE Transactions on Information Theory* 44.5 (Sep. 1998).
- [6] R. Karp. „Minimum-Redundancy Coding for the Discrete Noiseless Channel“. In: *IRE Transactions on Information Theory* 7.1 (Jan. 1961).
- [7] Prof. Martin Lercher. *Algorithm und Datenstrukturen*. Aufgerufen am 25.10.2025. 2016. URL: https://www.cs.hhu.de/fileadmin/redaktion/Fakultaeten/Mathematisch-Naturwissenschaftliche_Fakultaet/Informatik/Computational_Cell_Biology/AlDat/Alg2016_06.pdf.
- [8] Ralph-Hardo Schulz. *Codierungstheorie: Eine Einführung*. 2. Aufl. 2018. ISBN: 978-3-322-80328-3.